Support Vector Machines

Contents

9.1 Precursor: Method of Lagrange Multipliers **1**
9.1.1 Gradient ascent - unconstrained optimization . . . 2
9.1.2 Adding a single constraint . . . 2
9.1.3 Multiple constraints . . . 3

9.2 Maximum Margin Classifiers **5**
9.2.1 Identifying the margin . . . 5
9.2.2 Margins and the VC dimension . . . 7

9.3 The Support Vector Machine (SVM) **10**
9.3.1 SVMs are Maximum Margin Classifiers . . . 10
9.3.2 Structural risk minimization (SRM) . . . 11
9.3.2.1 Applying the Lagrange method to the primal problem of SRM . . . 12
9.3.3 Karush-Kuhn-Tucker (KKT) conditions . . . 13
9.3.4 Solving the primal problem . . . 14
9.3.5 From the primal problem to the dual problem . . . 15
9.3.6 The dual problem . . . 16
9.3.7 Making predictions with the SVM . . . 17
9.3.8 Non-linear separability . . . 18
9.3.8.1 Typical kernel functions . . . 18
9.3.8.2 SVM-RBF vs. RBF networks . . . 19

9.4 C-SVM **21**
9.4.1 From hard-margin to the soft-margin hyperplane . . . 21
9.4.1.1 The slack values . . . 22

9.5 References **25**

9.1 Precursor: Method of Lagrange Multipliers

Don't panic!

Essentially just hill-climbing (gradient ascent)

Objective: *Maximize* an objective function while also satisfying some constraint(s). (*Maximize*: Find the arguments that maximize an objective function.)

Don't forget: any maximization problem can be turned into a minimization problem by maximizing the "negative" of the function.

9.1.1 Gradient ascent - unconstrained optimization

Maximize an objective function $f(w_1, w_2)$ (no constraints here). Here w_1 and w_2 are referred to as free parameters.

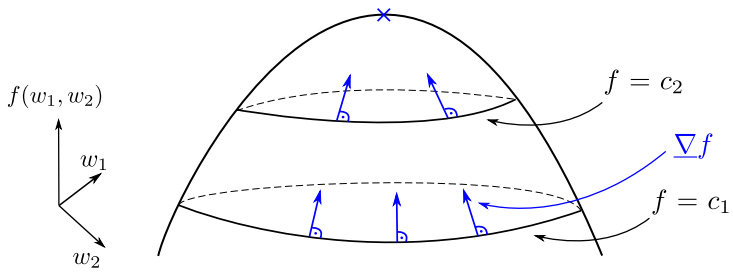


Figure 9.1.1: An unconstrained optimization problem.

Fig. 9.4.3 depicts the objective function f we want to maximize (i.e. reach the blue $\times$ at the top). c_1 and c_2 are level curves where f is constant. The blue arrows indicate the direction of the gradient $\underline{\nabla} f$, which is always perpendicular to the level curve.

The gradient $\underline{\nabla} f$ describes the direction of greatest ascent:

$$\underline{\nabla} f = \frac{\partial f}{\partial \underline{\mathbf{w}}} = \begin{pmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \end{pmatrix} \tag{9.1.1}$$

9.1.2 Adding a single constraint

We restrict solutions to those that satisfy the constraint $g(w_1, w_2) = c$:

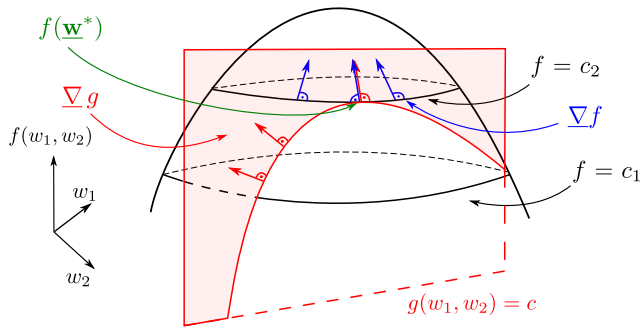


Figure 9.1.2: The surface of the constraint cuts through the surface of the objective function. The highest point that can be reached on f which also intersects with g is unique in that the gradients of f and g both point in the same direction with only a difference in scale.

Q 9.1.1: *What is characteristic of the solution $\underline{\mathbf{w}}^*$ to the constrained optimization problem?* -The solution of the constrained optimization problem is

characterized by:

$$\underline{\nabla} f\big|_{\underline{\mathbf{w}}^*} = \lambda' \underline{\nabla} g\big|_{\underline{\mathbf{w}}^*}, \quad \text{where } \lambda' \text{ is a scaling factor.} \tag{9.1.2}$$

Eq. (9.1.2) holds for the highest position in f while also satisfying the constraint.

Consequently,

$$\underline{\nabla} f \; - \lambda' \underline{\nabla} g = \underline{\mathbf{0}} \tag{9.1.3}$$

$$\underline{\nabla} f \; + \underbrace{(-\lambda')}_{=:\lambda} \underline{\nabla} g = \underline{\mathbf{0}} \tag{9.1.4}$$

$$\underline{\nabla} f \; + \lambda \underline{\nabla} g = \underline{\mathbf{0}} \; ^{*} \tag{9.1.5}$$

The constrained optimization problem is formulated as:

$$\underbrace{f(w_1, w_2) \; \stackrel{!}{=} \; \max_{w_1, w_2}}_{\text{maximization}} \quad \text{subject to} \quad \underbrace{g(w_1, w_2) \; - c \; = \; 0}_{\text{a single constraint}} \tag{9.1.6}$$

The *Lagrangian* function reformulates the constrained optimization problem from Eq. (9.1.6) in a way that reflects the relationship of the two gradients at the solution and thus facilitate finding the solution Eq.(9.1.5). The *Lagrangian* is therefore defined as:

$$L(w_1, w_2, \lambda) \; := \; f(w_1, w_2) + \lambda\big(g(w_1, w_2) - c\big) \tag{9.1.7}$$

Setting the gradient ∇L to zero guarantees a solution at which $\underline{\nabla} f \; + \lambda \underline{\nabla} g = \underline{\mathbf{0}}$

$$\underline{\nabla} L = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \\ \frac{\partial L}{\partial w_2} \\ \\ \frac{\partial L}{\partial \lambda} \end{pmatrix} = \begin{pmatrix} \frac{\partial f}{\partial w_1} & + & \lambda \frac{\partial g}{\partial w_1} \\ \frac{\partial f}{\partial w_2} & + & \lambda \frac{\partial g}{\partial w_2} \\ \underbrace{\frac{\partial f}{\partial \lambda}}_{=0} & + & g(w_1, w_2) - c \end{pmatrix} = \begin{pmatrix} 0 \\ \\ 0 \\ \\ 0 \end{pmatrix} = \underline{\mathbf{0}} \tag{9.1.8}$$

Setting the first two elements of $\underline{\nabla} L$, namely $\frac{\partial L}{\partial w_1}$ and $\frac{\partial L}{\partial w_2}$, to zero ensures that $\nabla f = -\lambda \nabla g$, while $\frac{\partial L}{\partial \lambda} = 0$ ensures that $g(w_1, w_2) = c$.

Eq.(9.1.8) describes a system of 3 equations with 3 unknowns. [2]

We refer to λ as the *multiplier* for the constraint c.

9.1.3 Multiple constraints

Q 9.1.2: *How do we extend this to multiple equality constraints?*

$$\underbrace{f_0(\underline{\mathbf{w}}) \; \stackrel{!}{=} \; \max}_{\text{maximization}} \quad \text{and} \quad f_k(\underline{\mathbf{w}}) = \; 0 \; , \quad k = 1, \ldots, m \tag{9.1.9}$$

where m denotes the number of constraints. $f_0(\underline{\mathbf{w}})$ is reserved for the function to be optimized., while $f_1(\underline{\mathbf{w}}), f_2(\underline{\mathbf{w}}), \ldots, f_m(\underline{\mathbf{w}})$ is used for all m constraints.

[*] The switch from λ' to λ is to be more consistent with the lecture slides.

[2] Not much emphasis is put on knowing how to solve this by hand. If interested, the Wikipedia article on Lagrange multiplier provides some examples.

The Lagrangian for multiple constraints is defined as:

$$L(\underline{\mathbf{w}}\,, \overbrace{\{\lambda_k\}}^{\substack{\text{a multiplier}\\ \text{for each constraint}}}) \;:=\; f_0(\underline{\mathbf{w}}) + \lambda_1\, f_1(\underline{\mathbf{w}}) + \lambda_2\, f_2(\underline{\mathbf{w}}) + \;\ldots\; + \lambda_m\, f_m(\underline{\mathbf{w}}) \tag{9.1.10}$$

$$= f_0(\underline{\mathbf{w}}) + \sum_{k=1}^{m} \lambda_k\, f_k(\underline{\mathbf{w}}) \tag{9.1.11}$$

Q 9.1.3: *What if we have inequality constraints?*

In the case of inequality constraints, our constrained optimization problem would have the following form:

$$\underbrace{f_0(\underline{\mathbf{w}}) \stackrel{!}{=} \max}_{\text{maximization}} \quad \text{and} \quad f_k(\underline{\mathbf{w}}) \leq 0\,, \quad k = 1, \ldots, m \tag{9.1.12}$$

However, the Lagrangian remains the same as in Eq. (9.1.11). The inequality is taken into account in that the solutions for λ extend to some range:

$$L(\underline{\mathbf{w}}\,, \{\lambda_k\}) \;:=\; f_0(\underline{\mathbf{w}}) + \sum_{k=1}^{m} \lambda_k\, f_k(\underline{\mathbf{w}})\,, \qquad \lambda_k \geq 0 \quad \forall k \in \{1, \ldots, m\} \tag{9.1.13}$$

9.2 Maximum Margin Classifiers

9.2.1 Identifying the margin

Consider the following linearly separable binary classification problem and the different hyperplanes for separating the two classes. In order to identify the margin of a hyperplane:

(a) select a hyperplane,

(b) select the training point closest to the hyperplane. Call it $\underline{\mathbf{x}}^*$. $\underline{\mathbf{x}}^*$ has the shortest normal distance to the hyperplane.

(c) measure the normal distance between $\underline{\mathbf{x}}^*$ and the hyperplane and call it d_w

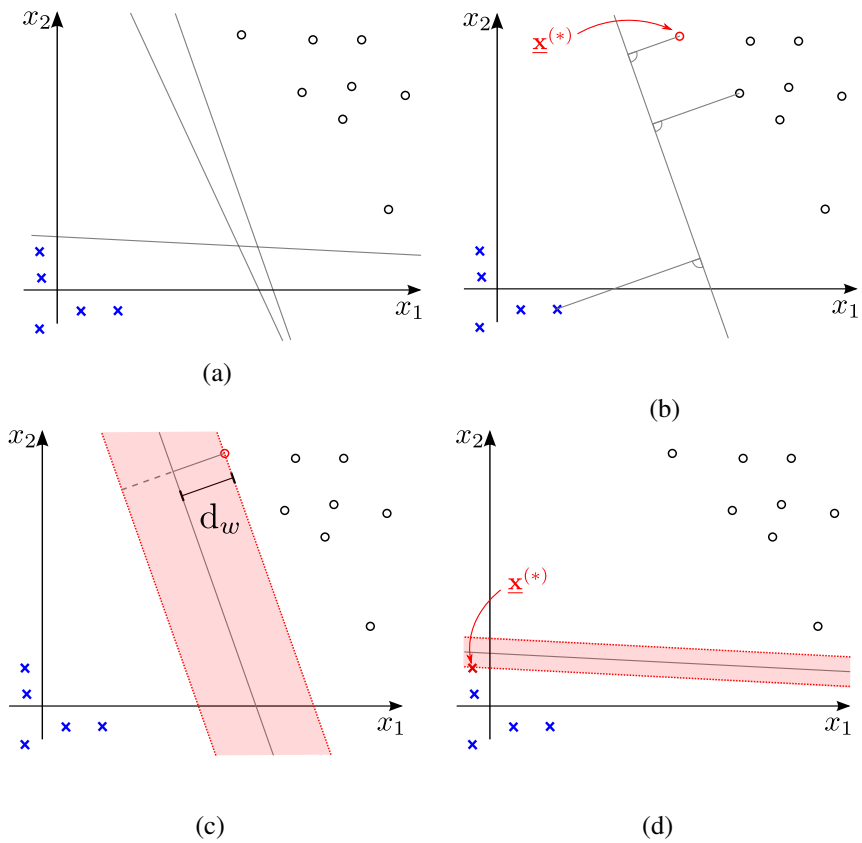


Figure 9.2.1: Identifying the margin d_w for a hyperplane.

The distance d_w is the *margin* for that hyperplane.

Q 9.2.1: *What are the implications of a wide vs. narrow margin?*

Remember: We are only dealing with a linearly separable problem.

The *margin* is the smallest distance between a separating hyperplane and a training sample. The training sample closest to the decision boundary is denoted with $\underline{\mathbf{x}}^{(*)}$. Separating the training data with a larger margin is more likely to reduce classification error on unseen data (i.e. lower generalization error, less overfitting). A simplistic justification for this: Assume we have found the separating hyperplane with the maximum margin for perfectly separable data. From this:

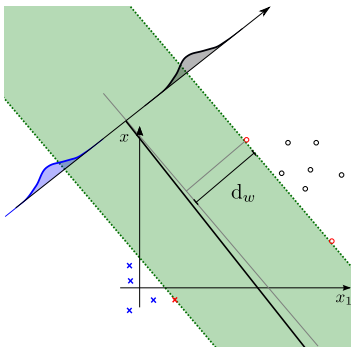


Figure 9.2.2: Potential generalization abilities with maximum margin.

(1) The distance between the classes is at least twice as wide as the distance from the hyperplane to $\underline{\mathbf{x}}^{(*)}$ (i.e. $2\mathrm{d}_w$).

(2) An unseen pattern $\underline{\mathbf{x}}^{(test)}$ is more likely to land closer to training patterns of the same class than patterns from the other class.

(3) The distance between $\underline{\mathbf{x}}^{(test)}$ and $\underline{\mathbf{x}}^{(*)}$ is more likely to be

- *larger* than the margin, if the points belong to *different* classes and more likely to be
- *below* the margin, if both belong to the *same* class.

Another way to look at this:

Consider a second hyperplane with the same orientation $\underline{\mathbf{w}}$ but less than maximum (i.e. more narrow) margin. Shift the second hyperplane so that our sample $\underline{\mathbf{x}}^{(*)}$ also lies on the margin of the second hyperplane. The shifting results in the decision boundaries of both hyperplanes appearing parallel to one another, both make contact with $\underline{\mathbf{x}}^{(*)}$. The difference is that the hyperplane with the smaller margin appears to lie inside the first hyperplane. Now consider an unseen pattern $\underline{\mathbf{x}}_{(test)}$ that is very similar to $\underline{\mathbf{x}}^{(*)}$ (i.e. $\underline{\mathbf{x}}_{(test)} = \underline{\mathbf{x}}^{(*)} + \underline{\mathbf{noise}}$). $\underline{\mathbf{x}}_{(test)}$ belongs to the same class as $\underline{\mathbf{x}}^{(*)}$. The noise can result in the $\underline{\mathbf{x}}_{(test)}$ falling inside the margin of the first hyperplane. Because the margin of the second hyperplane is more narrow, the decision boundary of that plane is closer to $\underline{\mathbf{x}}^{(*)}$. This means that $\underline{\mathbf{x}}_{(test)}$ is more likely to fall on the wrong side of the second decision boundary and get misclassified (i.e. higher generalization error). This is less likely to happen for the first hyperplane because the maximum margin pushes the decision boundary away from the $\underline{\mathbf{x}}^{(*)}$. We would need to apply more noise on $\underline{\mathbf{x}}_{(test)}$ until it gets misclassified by the maximum margin hyperplane.

Another implication of the maximizing the margin is that the optimal hyperplane with maximum margin is a *unique* solution to the optimization problem.

9.2.2 Margins and the VC dimension

Q 9.2.2: *Where does the VC dimension come into all of this?*

The short answer:

$$\uparrow \text{margin} \quad \rightsquigarrow \quad \mathrm{d}_{\mathrm{VC}} \downarrow \tag{9.2.1}$$

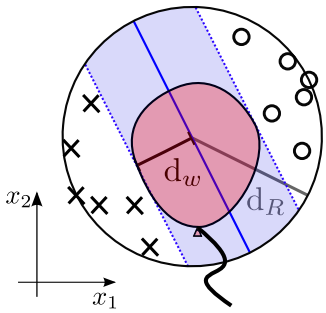


Figure 9.2.3: The general setting assumed for deriving the upper bound on the VC dimension of the maximum margin classifier.

Large margins imply a small VC dimension. A larger margin tightens the upper bound of the VC dimension, potentially disregarding the dimensionality of the problem.

***Theorem 10.3* in (Vapnik, 1998)**

states that

$$\mathrm{d}_{\mathrm{VC}} \quad \leq \quad \min\left(\left\lfloor \frac{\mathrm{d}_R^2}{\mathrm{d}_w^2} \right\rfloor, N\right) + 1 \tag{9.2.2}$$

where,

N : dimension of feature space

d_w : the margin: $\frac{1}{\|\underline{\mathbf{w}}\|} \geq \mathrm{d}_w$

d_R : radius of a sphere containing all training points.
It quantifies the support of $\underline{\mathbf{x}}$:
$P(\underline{\mathbf{x}}) \neq 0$ for $\|\underline{\mathbf{x}}\| \leq \mathrm{d}_R$ (i.e. no points exist outside of this sphere.)

The bound in Eq.(9.2.2) assumes a setting as depicted by Fig. 9.2.3. It tells us that, if we draw a sphere around our training data with radius d_R and a second sphere inside of it using the margin d_w as its radius, knowing that the inner sphere is placed in the center and does not contain any datapoints, the theorem then tells us that the VC dimension is no longer $N + 1$ but only as big as the integer component [3] [4] on the ratio $\frac{\mathrm{d}_R^2}{\mathrm{d}_w^2}$ and $+1$.

To illustrate:

Recall the definition of the d_{VC} as the maximal number of training points p that a model can *shatter*.

[3] If interested in a proof for *Theorem 10.3*, see http://web.eecs.umich.edu/~jabernet/eecs598course/fall2015/web/not

[4] According to (Hush and Scovel, 2001), it is debatable whether one should apply $\lfloor\cdot\rfloor$ (floor) or $\lceil\cdot\rceil$ (ceil operation) while pointing out that the latter is possibly incorrect but "asymptotically sharper" with reference to (Burges, 1998).

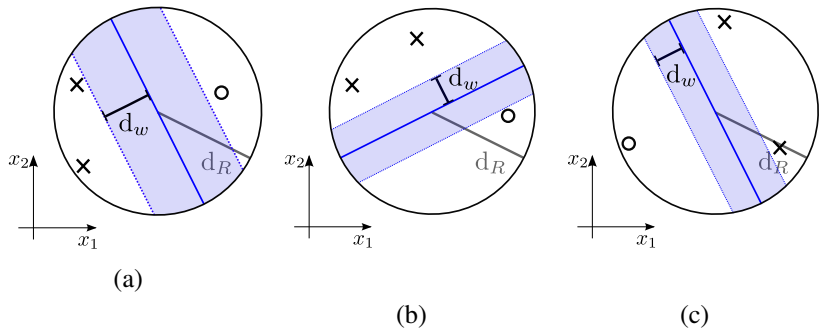


Figure 9.2.4: A linear neuron can shatter any label configuration of 3 points regardless of how they are arranged in 2D space (excluding collinear points).

Therefore, $\mathrm{d}_{\mathrm{VC}} = N + 1 = 3$, for a linear classifier with a margin $0 \leq \mathrm{d}_w < \mathrm{d}_R$. As the margin increases, i.e. $\mathrm{d}_w \to \mathrm{d}_R$, d_{VC} of the linear classifier drops from $N + 1$ to $1 + 1 = 2$ (e.g. from 3 to 2). In the case of $\mathrm{d}_w = \mathrm{d}_R$ the VC dimension will be remains $1 + 1 = 2$. However it is a case where there is no more space that is inside the sphere and outside of the margin as well. Without this makes this case not particulary interesting to us.

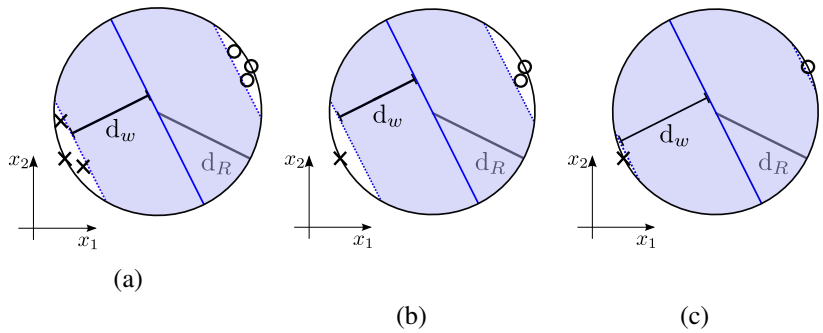


Figure 9.2.5: A linear neuron can no longer shatter $N + 1$ points when the margin is maximal.

If the margin provides a bound for the d_{VC}

Q 9.2.3: *What were the benefits of reducing* d_{VC}*?* Recall that one of the key

results of Statistical Learning Theory was the formulation of the bound on the generalization error, given a finite set of samples:

$$P\left\{ \sup_{\underline{\mathbf{w}} \in \Lambda} \left| R_{[\underline{\mathbf{w}}]} - R^{(p)}_{\mathrm{emp}[\underline{\mathbf{w}}]} \right| > \eta \right\} < 4 \exp\left(G^{\Lambda}_{(2p)} - p\left(\eta - \tfrac{1}{p}\right)^2 \right) \stackrel{!}{=} \epsilon \quad (9.2.3)$$

where Λ denotes the model class and $G^{\Lambda}_{(2p)}$ denotes the growth function[5].

With a probability larger than $1 - \epsilon$ we obtain:

$$R_{[\underline{\mathbf{w}}]} < \underbrace{R^{(p)}_{\mathrm{emp}[\underline{\mathbf{w}}]}}_{\substack{\text{empirical}\\ \text{error}}} + \underbrace{\left(\frac{G^{\Lambda}_{(2p)} - \ln \frac{\epsilon}{4}}{p} \right)^{\frac{1}{2}} + \frac{1}{p}}_{\text{complexity term } C(p, \mathrm{d}_{\mathrm{VC}})} \quad (9.2.4)$$

[5] cf. lecture 2.1.3 on Statistical Learning Theory and corresponding notes.

For a given ϵ, the complexity term only depends on p and $\mathrm{d_{VC}}$. Therefore, the VC dimensions helps in formulating a bound on the **generalization ability** of Empirical Risk Minimization (ERM). Knowing that

- the risk $R_{[\underline{\mathbf{w}}]}$ and empirical risk $R_{\text{emp}[\underline{\mathbf{w}}]}$ are represented by the generalization $E^G_{[\underline{\mathbf{w}}]}$ and training error $E^T_{[\underline{\mathbf{w}}]}$, respectively:

$$R_{[\underline{\mathbf{w}}]} \mathrel{\widehat{=}} E^G_{[\underline{\mathbf{w}}]}\,,$$
$$R_{\text{emp}[\underline{\mathbf{w}}]} \mathrel{\widehat{=}} E^T_{[\underline{\mathbf{w}}]}$$

  and that

- for $p > \mathrm{d_{VC}}$:

$$G^{\Lambda}_{(p)} \leq \mathrm{d_{VC}}\Big(\ln \frac{p}{\mathrm{d_{VC}}} + 1\Big)\,,$$

Eq.(9.2.4) can be reformulated in order to highlight the relationship between the VC dimension and the generalization ability of the model class:

For a finite training set, the generalization error is bound with probability $1-\epsilon$:

$$E^G_{[\underline{\mathbf{w}}]} \;\leq\; E^T_{[\underline{\mathbf{w}}]} + \underbrace{\sqrt{\frac{\mathrm{d_{VC}}\left(\ln \frac{2p}{\mathrm{d_{VC}}} + 1\right) - \ln\frac{\epsilon}{4}}{p}}}_{\text{generalization gap}} + \frac{1}{p} \quad \text{for } p > \mathrm{d_{VC}} \qquad (9.2.5)$$

Q 9.2.4: *What about* $\mathrm{d_{VC}} \to 0$*? Is this ideal?*

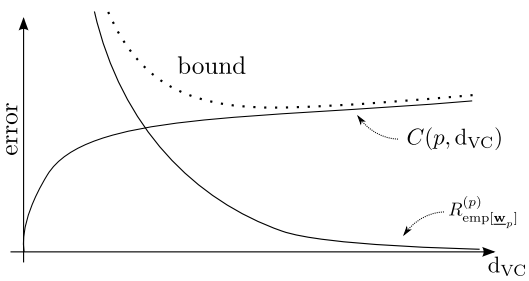


Figure 9.2.6: The effect of the VC dimension and bias-variance trade-off

9.3 The Support Vector Machine (SVM)

9.3.1 SVMs are Maximum Margin Classifiers

The binary classification setting:

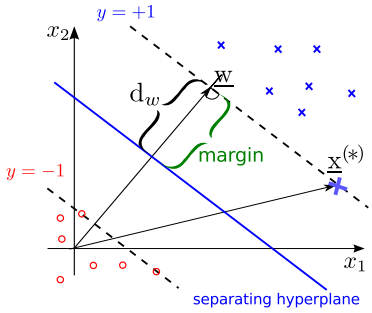


Figure 9.3.1: Binary classification setting.

- Data:
  $\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}_{\alpha=1}^{p}$ where $\underline{\mathbf{x}} \in \mathbb{R}^N$, $y_T \in \{-1, 1\}$
- Model:
  $y(\underline{\mathbf{x}}; \underline{\mathbf{w}}, b) = \operatorname{sign}\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}} + b \right)$
- Objective:
  Choose the decision boundary s.t. the margin is *maximal*.
- Assumption:
  Perfect linear separation of both classes.
  In the case of a simple connectionist neuron:

$$\underline{\mathbf{w}}^\top \underline{\mathbf{x}} + b \begin{cases} \geq & 0 \text{ for } y_T = +1 \\ < & 0 \text{ for } y_T = -1 \end{cases} \tag{9.3.1}$$

  The solution here is not unique.

Let $\underline{\mathbf{w}}^*$ and b^* describe the optimal hyperplane that maximizes the margin of separation:

$$\underline{\mathbf{w}}^{*\top} \underline{\mathbf{x}} + b^* \begin{cases} \geq & 1 & \text{for } y_T = +1 \\ \leq & -1 & \text{for } y_T = -1 \end{cases} \tag{9.3.2}$$

Points that lie on the margin are characterized by:

$$\underline{\mathbf{w}}^{*\top} \underline{\mathbf{x}} + b^* \begin{cases} = & 1 & \text{for } y_T = +1 \\ = & -1 & \text{for } y_T = -1 \end{cases} \tag{9.3.3}$$

$\longleftarrow$ "support vectors"

These points will be referred to as the *support vectors*. The shortest distance between points with opposite class labels is $2\mathrm{d}_w = \frac{2}{\|\underline{\mathbf{w}}^*\|_2}$, i.e. the distance separating the two classes.

Maximizing the margin $\widehat{=}$ minimizing the Euclidean norm of $\underline{\mathbf{w}}$[6].

[6] For more details on the geometric intepretation of the margin and how maximizing it relates to minimizing $\|\underline{\mathbf{w}}\|_2$, see Scholkopf and Smola, 2001, Chapters 7.1 and 7.2.

$$\mathrm{d}_w = \frac{1}{\|\underline{\mathbf{w}}\|_2} \stackrel{!}{=} \max \quad \widehat{=} \quad \|\underline{\mathbf{w}}\|_2 \stackrel{!}{=} \min \tag{9.3.4}$$

The solution to this is unique.

9.3.2 Structural risk minimization (SRM)

The cost function to minimize:

$$\frac{1}{2}\underline{\mathbf{w}}^\top \underline{\mathbf{w}} = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 \stackrel{!}{=} \min \tag{9.3.5}$$

Q 9.3.1: *Why have we suddenly switched to the squared Euclidean norm and where did the* $1/2$ *come from?* - Minimizing the square of the norm

turns it into a *convex* minimization problem, which simplifies the optimization procedure. Dividing by 2 is simply for convenience.

We want to restrict the space of solutions for $\underline{\mathbf{w}}$ to that which leads to

1. the correct classification of the training points.
2. training points fall on or outside the margin, but **not in**side the margin (see Table 1 for example cases).

We therefore constrain solutions such that:

$$y_T^{(\alpha)} \cdot \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) \geq 1 \quad \forall \alpha \tag{9.3.6}$$

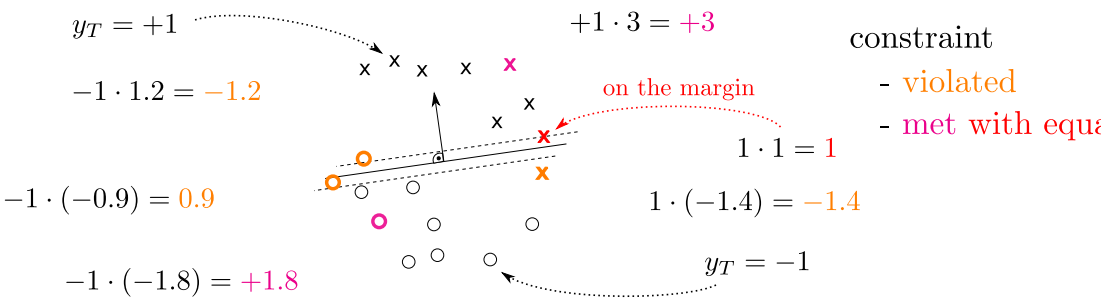


Figure 9.3.2: Example cases for meeting or violating the constraint. The constraint is violated whenever a point is misclassified or falls inside the margin. It is met whenever it is correctly classified and falls outside of the margin and met with equality whenever it is correctly classifiied and falls exactly on the margin

Eq.(9.3.6) provides constraints that are linear in $\underline{\mathbf{w}}$. It builds on the *0-1 loss* or *hinge loss*. In its original form, the *0-1 loss* is exactly zero whenever a point is correctly classified and 1 whenever it is misclassified. It behaves similarly to *parity-check* with the difference that *parity-check* only produces a binary output. We modify the *0-1 loss* to also detect whether a point has fallen inside a margin.

This gives us the *primal* problem for structural risk minimization (SRM) problem.

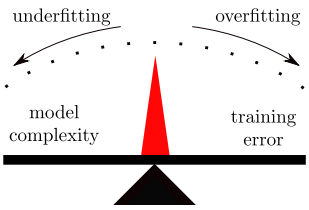


Figure 9.3.3: The SRM principle is about balancing the complexity of a model while minimizing the training error to avoid overfitting.

9.3.2.1 Applying the Lagrange method to the primal problem of SRM

$$f_0(\underline{\mathbf{w}}, b) = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top \underline{\mathbf{w}} \stackrel{!}{=} \min \tag{9.3.7}$$

Manipulating the constraints Eq. (9.3.6) to obtain an expression in the form $f_\alpha \leq 0$:

A constraint $f_\alpha(\underline{\mathbf{w}}, b) \leq 0$ for **each data point** in the training set:

$$y_T^{(\alpha)} \cdot \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) \geq 1 \tag{9.3.8}$$

$$y_T^{(\alpha)} \cdot \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) - 1 \geq 0 \tag{9.3.9}$$

$$-\left\{y_T^{(\alpha)} \cdot \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) - 1\right\} \leq 0 \tag{9.3.10}$$

$$f_\alpha(\underline{\mathbf{w}}, b) = -\left\{y_T^{(\alpha)} \cdot \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) - 1\right\} \leq 0\,, \quad \alpha = 1, \ldots, p \tag{9.3.11}$$

The Lagrangian becomes:

$$L(\underline{\mathbf{w}}, b, \{\lambda_\alpha\}) = f_0(\underline{\mathbf{w}}, b) + \sum_{\alpha=1}^{p} \lambda_\alpha f_\alpha(\underline{\mathbf{w}}, b) \tag{9.3.12}$$

$$= \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 - \sum_{\alpha=1}^{p} \lambda_\alpha \left\{y_T^{(\alpha)} \cdot \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) - 1\right\} \tag{9.3.13}$$

where $\{\lambda_a \geq 0\}_{\alpha=1}^{p}$ is the set of Lagrange multipliers. Because of the minus sign in front of the multipliers, the Lagrangian is optimized by

minimizing w.r.t. $\underline{\mathbf{w}}$ and b, the *"primal variables"*, and

maximizing w.r.t. all λ_α, the *"dual variables"*.

This, along with the minimization being *convex* and the constraints being linear, implies that a *saddle point* has to be found for the Lagrangian.

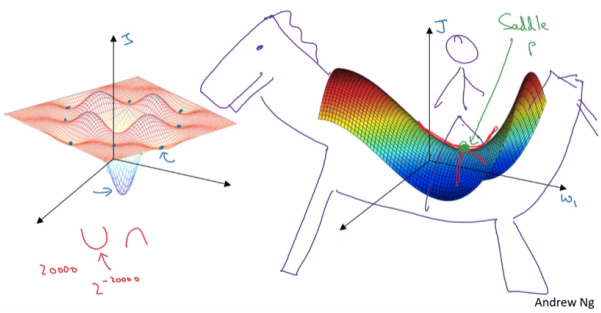


Figure 9.3.4: Illustration of a saddle point.

Equivalent optimization problems (Kuhn and Tucker theorem):

$$\min_{\underline{\mathbf{w}},b} L(\underline{\mathbf{w}}, b, \{\lambda_\alpha^*\}) = L(\underline{\mathbf{w}}^*, b^*, \{\lambda_\alpha^*\}) = \max_{\lambda_\alpha \geq 0} L(\underline{\mathbf{w}}^*, b^*, \{\lambda_\alpha\}) \quad (9.3.14)$$

9.3.3 Karush-Kuhn-Tucker (KKT) conditions

Q 9.3.2: *What is happening as we optimize the Lagrangian of the primal problem?*

- We will attempt to answer this by looking at the constraints and how different cases for the constraints affect the Lagrangian and in turn the variables we are optimizing. This will leads us to identifying the Karush-Kuhn-Tucker (KKT) conditions.

Recall

1. the constrained optimization problem:

$$\begin{array}{ll} \min_{\underline{\mathbf{w}},b} & \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 \\ \text{subject to} & -\{y_T^{(\alpha)} \cdot (\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b) - 1\} \leq 0\,, \quad \text{for all } \alpha = 1, \ldots, p, \end{array}$$

2. the corresponding Lagrangian:

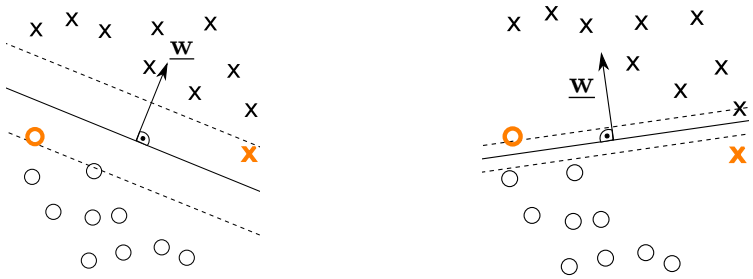
$$L(\underline{\mathbf{w}}, b, \{\lambda_\alpha\}) = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 - \sum_{\alpha=1}^{p} \lambda_\alpha \{y_T^{(\alpha)} \cdot (\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b) - 1\}, \quad \lambda_\alpha \geq 0,$$

3. and the equivalent optimization problems:

$$\min_{\underline{\mathbf{w}},b} L(\underline{\mathbf{w}}, b, \{\lambda_\alpha^*\}) = L(\underline{\mathbf{w}}^*, b^*, \{\lambda_\alpha^*\}) = \max_{\lambda_\alpha \geq 0} L(\underline{\mathbf{w}}^*, b^*, \{\lambda_\alpha\}).$$

Consider the following cases:

(a) Whenever a constraint is violated, i.e. $-\{y_T^{(\alpha)} \cdot (\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b) - 1\} > 0$



(a) No points within the margin! (b) No misclassifications!

Figure 9.3.5: Possible causes of violating the constraint.

Increasing the value of the corresponding multiplier λ_α *increase* L. This will in turn lead to a change in the primal variables $\underline{\mathbf{w}}$ and b to *decrease* L. That is:

$$\lambda_\alpha \uparrow \rightsquigarrow \quad L\uparrow \;\rightsquigarrow\; \text{change in } \underline{\mathbf{w}}, b \;\rightsquigarrow\; L\downarrow \quad (9.3.15)$$

Decrease of L by changing $\underline{\mathbf{w}}$ and b prevents terms inside the sum of the Lagrangian from taking on arbitrarily large negative values.

(b) Constraints are met but **not** as equalities, i.e. $-\{y_T^{(\alpha)} \cdot (\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b) - 1\} < 0$. λs are meant to maximize L. In this case, L can only be maximized by setting the corresponding multiplier λ_α to zero.

(c) This leaves $\lambda_\alpha \neq 0$, which corresponds to the constraints being met with *equality*, i.e. $-\{y_T^{(\alpha)} \cdot (\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b) - 1\} \stackrel{!}{=} 0$.

Q 9.3.3: *What does "=0" imply w.r.t. misclassification?*

-This implies that the choice of $\underline{\mathbf{w}}$ and b not only leads to correct classification of that training point but also that the point lies exactly on the margin, indicating that the point for which $\lambda_\alpha \neq 0$ is a *support vector* (cf. Fig. 9.3.1). These are referred to as the *Karush-Kuhn-Tucker (KKT) conditions*.

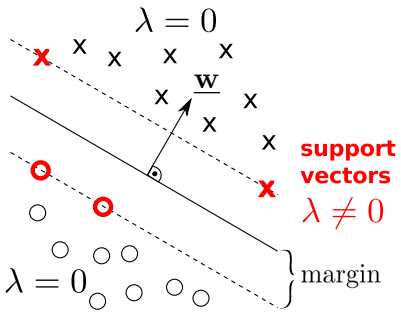


Figure 9.3.6: The constraint for support vectors is met with equality.

Where is the point relative to the margin?	label y_T	prediction $\underline{\mathbf{w}}^\top \underline{\mathbf{x}} + b$	correctly classified	constraint value f_α	constraint met?
outside	+1	-11	no	-{+1·(-11) - 1} = +12 > 0	violated
outside	-1	+30	no	-{-1·30 - 1} = +31 > 0	violated
inside	+1	+0.2	yes	-{+1·0.2 - 1} = +0.8 > 0	violated
inside	-1	-0.2	yes	-{-1·(-0.2) - 1} = +0.8 > 0	violated
outside	+1	+200	yes	–{+1·200 - 1} = -{199} < 0	met
outside	-1	-10	yes	-{-1(-10) - 1} = -{9} < 0	met
on	-1	-1	yes	-{-1·(-1) - 1} = 0	met with equality

Table 1: Example cases for the constraint.

9.3.4 Solving the primal problem

The Lagrangian of the primal problem is solved by finding the saddle point. At the saddle point, the derivatives of the Lagrangian L w.r.t to the primal variables goes to zero:

$$\frac{\partial L}{\partial w_i} \stackrel{!}{=} 0 \quad \text{for } i = 1, \ldots, N \tag{9.3.16}$$

$$w_i - \sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} x_i^{(\alpha)} \stackrel{!}{=} 0 \tag{9.3.17}$$

$$\Rightarrow \quad \underline{\mathbf{w}}^* = \underbrace{\sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \underline{\mathbf{x}}^{(\alpha)}}_{\substack{\text{expansion of the weight vector} \\ \text{in terms of data points}}} \tag{9.3.18}$$

$$\frac{\partial L}{\partial b} = -\sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \stackrel{!}{=} 0 \tag{9.3.19}$$

$$\sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \stackrel{!}{=} 0 \tag{9.3.20}$$

The solution is unique for $\underline{\mathbf{w}}$ but not for the set of multipliers $\{\lambda_a\}_{\alpha=1}^{p}$ (we haven't forgotten about b^*, we will get to b^* shortly).

9.3.5 From the primal problem to the dual problem

The *dual* problem:

Solving the primal problem in addition to finding a unique solution for the set of multipliers.

Eq. (9.3.18) and Eq. (9.3.20) are plugged back into the Lagrangian of the primal problem Eq.(9.3.13). From this follows:

$$L(\underline{\mathbf{w}}^*, b, \{\lambda_\alpha\}) \tag{9.3.21}$$

$$= \frac{1}{2} \underline{\mathbf{w}}^{*\top} \underline{\mathbf{w}}^* - \sum_{\alpha=1}^{p} \lambda_\alpha \{ y_T^{(\alpha)} \cdot (\underline{\mathbf{w}}^{*\top} \underline{\mathbf{x}}^{(\alpha)} + b) - 1 \} \tag{9.3.22}$$

$$= \frac{1}{2} \underline{\mathbf{w}}^{*\top} \underline{\mathbf{w}}^* - \sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \underline{\mathbf{w}}^{*\top} \underline{\mathbf{x}}^{(\alpha)} - \sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} b + \sum_{\alpha=1}^{p} \lambda_\alpha \tag{9.3.23}$$

$$= \frac{1}{2} \underline{\mathbf{w}}^{*\top} \underline{\mathbf{w}}^* - \underline{\mathbf{w}}^{*\top} \underbrace{\sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \underline{\mathbf{x}}^{(\alpha)}}_{=\underline{\mathbf{w}}^* \text{ (cf. Eq(9.3.18))}} - b \underbrace{\sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)}}_{=0 \text{ (cf. Eq(9.3.20))}} + \sum_{\alpha=1}^{p} \lambda_\alpha \tag{9.3.24}$$

$$= \frac{1}{2} \underline{\mathbf{w}}^{*\top} \underline{\mathbf{w}}^* - \underline{\mathbf{w}}^{*\top} \underline{\mathbf{w}}^* + \sum_{\alpha=1}^{p} \lambda_\alpha \tag{9.3.25}$$

$$= -\frac{1}{2} \underline{\mathbf{w}}^{*\top} \underline{\mathbf{w}}^* + \sum_{\alpha=1}^{p} \lambda_\alpha \tag{9.3.26}$$

plugging in the solution Eq.(9.3.18) for $\underline{\mathbf{w}}^*$:

$$= -\frac{1}{2} \sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \underline{\mathbf{x}}^{(\alpha)\top} \sum_{\beta=1}^{p} \lambda_\beta y_T^{(\beta)} \underline{\mathbf{x}}^{(\beta)} + \sum_{\alpha=1}^{p} \lambda_\alpha \tag{9.3.27}$$

$$L(\{\lambda_\alpha\}) = -\frac{1}{2} \sum_{\alpha=1}^{p} \sum_{\beta=1}^{p} \lambda_\alpha \lambda_\beta y_T^{(\alpha)} y_T^{(\beta)} \underline{\mathbf{x}}^{(\alpha)\top} \underline{\mathbf{x}}^{(\beta)} + \sum_{\alpha=1}^{p} \lambda_\alpha \stackrel{!}{=} \max_{\{\lambda_\alpha\}} \tag{9.3.28}$$

Eq. (9.3.28) gives us the maximization objective of the *dual problem*. Note that the Lagrangian of the dual problem is a function of the set of multipliers $\{\lambda_\alpha\}$ only.

9.3.6 The dual problem

Find the Lagrange multipliers $\{\lambda_\alpha\}$ that maximize Eq.(9.3.28):

$$L(\underline{\mathbf{w}}^*, b, \{\lambda_\alpha\}) = -\frac{1}{2}\sum_{\alpha=1}^{p}\sum_{\beta=1}^{p}\lambda_\alpha\lambda_\beta y_T^{(\alpha)} y_T^{(\beta)} \underline{\mathbf{x}}^{(\alpha)\top}\underline{\mathbf{x}}^{(\beta)} + \sum_{\alpha=1}^{p}\lambda_\alpha \stackrel{!}{=} \max_{\{\lambda_\alpha\}}$$

subject to

$\lambda_\alpha \geq 0, \quad \alpha = 1, \ldots, p$ and

$\sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} = 0$

Reuse the solution of the primal problem for $\underline{\mathbf{w}}^*$:

This dual problem is optimized numerically using *Sequential Minimal Optimization*.

Q 9.3.4: *What about the bias?*

By now we have:

- The orientation of the optimal hyperplane with maximum margin: $\underline{\mathbf{w}}^*$
- The set of multipliers $\{\lambda_\alpha\}$. *How can those help in finding the bias?*

The clue is in the constraints $f_\alpha(\underline{\mathbf{w}}^*, b^*) = -\left\{y_T^{(\alpha)}\left(\underline{\mathbf{w}}^{*\top}\underline{\mathbf{x}}^{(\alpha)} + b^*\right) - 1\right\} \quad \leq \quad 0$

Which is met with equality by any of the support vector:

$$f_\alpha(\underline{\mathbf{w}}^*, b^*) = -\left\{y_T^{(\alpha)}\left(\underline{\mathbf{w}}^{*\top}\underline{\mathbf{x}}^{(\alpha)} + b^*\right) - 1\right\} \stackrel{!}{=} 0 \tag{9.3.29}$$

Solving for b^* yields:

$$\Rightarrow \quad b^* = y_T^{(\alpha)} \quad - \quad \underline{\mathbf{w}}^{*\top}\underline{\mathbf{x}}^{(\alpha)} \tag{9.3.30}$$

The calculation of b^* is made more robust by averaging over all support vectors:

$$b^* = \frac{1}{\#\mathrm{SV}}\sum_{\alpha\in\mathrm{SV}}\left(y_T^{(\alpha)} - \sum_{\beta\in\mathrm{SV}}\lambda_\beta y_T^{(\beta)}\left(\underline{\mathbf{x}}^{(\beta)}\right)^\top\underline{\mathbf{x}}^{(\alpha)}\right) \tag{9.3.31}$$

Q 9.3.5: *Why does the inner sum run through the SVs only?* - Indeed,

$\underline{\mathbf{w}}^*$ is an expansion of the weight vector in terms of all data points (cf. Eq. (9.3.18)). However, any point that is **not** a support vector is associated with a λ that is equal to zero. Non-zero λ's are only associated with support vectors. Therefore, the sum can be reduced to iterate over the non-zero λ's only, which are those associated with the support vectors.

9.3.7 Making predictions with the SVM

1. We started off with a connectionist neuron model:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}, b) = \text{sign}\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}} + b\right), \tag{9.3.32}$$

2. optimized it to be a *maximum margin classifier*:

$$L(\{\lambda_\alpha\}) \;=\; -\frac{1}{2}\sum_{\alpha,\beta=1}^{p} \lambda_\alpha \lambda_\beta y_T^{(\alpha)} y_T^{(\beta)} \left(\underline{\mathbf{x}}^{(\alpha)}\right)^\top \underline{\mathbf{x}}^{(\beta)} + \sum_{\alpha=1}^{p} \lambda_\alpha \stackrel{!}{=} \max_{\{\lambda_\alpha\}} \tag{9.3.33}$$

$\{\lambda_\alpha \geq 0\}_{\alpha=1}^{p}\,, \quad \sum_{\alpha=1}^{p} \lambda_\alpha\, y_T^{(\alpha)} = 0$

3. This gave us the optimal parameters for our model:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}^*, b^*) = \text{sign}\left(\underline{\mathbf{w}}^{*\top} \underline{\mathbf{x}} + b^*\right), \tag{9.3.34}$$

with

$$\underline{\mathbf{w}}^* = \sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \underline{\mathbf{x}}^{(\alpha)} = \sum_{\beta \in \text{SV}} \lambda_\beta y_T^{(\beta)} \underline{\mathbf{x}}^{(\beta)} \tag{9.3.35}$$

and

$$b^* = \frac{1}{\#_{\text{SV}}} \sum_{\alpha \in \text{SV}} \left(y_T^{(\alpha)} - \sum_{\beta \in \text{SV}} \lambda_\beta y_T^{(\beta)} \left(\underline{\mathbf{x}}^{(\beta)}\right)^\top \underline{\mathbf{x}}^{(\alpha)}\right)$$

This reveals that our SVM classifier is essentially described by the SVs found in the training set. $\underline{\mathbf{w}}^*$ and b^* can be calculated directly from the SVs.

9.3.8 Non-linear separability

Q 9.3.6: *What about non-linearly separable classes?*

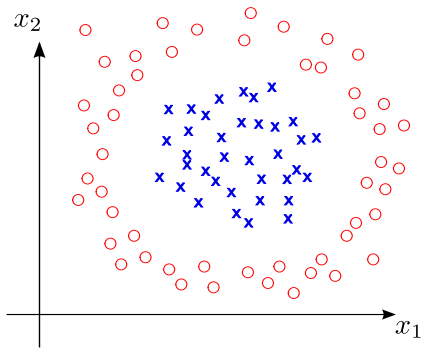


Figure 9.3.7: A non-linearly separable case.

- Feature transformation. The feature space is transformed into a new space in which the classes can become linearly separable:

$$\underbrace{\underline{\mathbf{x}}}_{\substack{\text{elementary}\\ \text{features}}} \longrightarrow \underbrace{\underline{\phi}_{(\underline{\mathbf{x}})}}_{\substack{\text{monomials of}\\ \text{degree } n}} \tag{9.3.36}$$

However, instead of computing the transformation $\underline{\phi}$ explicitly (possibly very costly in terms of memory footprint if the space of $\underline{\phi}$ has a very high dimensionality), we exploit the fact that SVMs only ever operate with the scalar product. This is evident from the Lagrangian of the dual problem we use to construct the SVM:

$$L(\{\lambda_\alpha\}) = -\frac{1}{2}\sum_{\alpha,\beta=1}^{p} \lambda_\alpha \lambda_\beta y_T^{(\alpha)} y_T^{(\beta)} \underbrace{\left(\underline{\mathbf{x}}^{(\alpha)}\right)^\top \underline{\mathbf{x}}^{(\beta)}}_{\circledast} + \sum_{\alpha=1}^{p} \lambda_\alpha \stackrel{!}{=} \max_{\{\lambda_\alpha\}} \tag{9.3.37}$$

A transformation of the features would result in:

$$L(\{\lambda_\alpha\}) = -\frac{1}{2}\sum_{\alpha,\beta=1}^{p} \lambda_\alpha \lambda_\beta y_T^{(\alpha)} y_T^{(\beta)} \underbrace{\left(\underline{\phi}_{\underline{\mathbf{x}}^{(\alpha)}}\right)^\top \underline{\phi}_{\underline{\mathbf{x}}^{(\beta)}}}_{\circledast} + \sum_{\alpha=1}^{p} \lambda_\alpha \stackrel{!}{=} \max_{\{\lambda_\alpha\}} \tag{9.3.38}$$

9.3.8.1 Typical kernel functions

$k_{(\underline{\mathbf{x}},\underline{\mathbf{x}}')} = \left(\underline{\mathbf{x}}^\top \underline{\mathbf{x}}' + 1\right)^d$	polynomial kernel of degree d $\rightarrow$ image processing: pixel correlations
$k_{(\underline{\mathbf{x}},\underline{\mathbf{x}}')} = \exp\left\{-\frac{(\underline{\mathbf{x}}-\underline{\mathbf{x}}')^2}{2\sigma^2}\right\}$	RBF-kernel with range σ $\rightarrow$ infinite dimensional feature space
$k_{(\underline{\mathbf{x}},\underline{\mathbf{x}}')} = \tanh\left\{\kappa \underline{\mathbf{x}}^\top \underline{\mathbf{x}}' + \theta\right\}$	neural network kernel with parameters κ and $\theta \rightarrow$ not positive definite
$k_{(\underline{\mathbf{x}},\underline{\mathbf{x}}')} = \frac{1}{(\lVert\underline{\mathbf{x}}-\underline{\mathbf{x}}'\rVert^2+\epsilon^2)^{N/2}}$	Plummer kernel with parameter ϵ $\rightarrow$ scale invariant kernel

$$k(\underline{\mathbf{x}}, \underline{\mathbf{x}}') \quad = \quad \sum_{i=1}^{\infty} \underbrace{\psi_{i(\underline{\mathbf{x}})} \sqrt{\lambda_i}}_{\phi_{i(\underline{\mathbf{x}})}} \underbrace{\sqrt{\lambda_i}\, \psi_{i(\underline{\mathbf{x}}')}}_{\phi_{i(\underline{\mathbf{x}}')}} \quad = \quad \underline{\phi}_{(\underline{\mathbf{x}})}^{\top} \underline{\phi}_{(\underline{\mathbf{x}}')}$$

- every positive semi-definite kernel K is an **inner product** in the induced space Φ, spanned by the features $\phi_{i(\underline{\mathbf{x}})} = \sqrt{\lambda_i}\, \psi_{i(\underline{\mathbf{x}})}$

- Φ is often high dimensional

- a linear classifier in Φ can solve non-linearly separable problems in the original feature space $\mathcal{X}$

The Lagrangian of the dual problem for transformed features involves computing $\left(\underline{\phi}_{\underline{\mathbf{x}}^{(\alpha)}}\right)^{\top} \underline{\phi}_{\underline{\mathbf{x}}^{(\beta)}}$ for all $\alpha, \beta = 1, \ldots, p$, i.e. all pairs of training points:

$$L(\{\lambda_\alpha\}) \quad = \quad -\frac{1}{2} \sum_{\alpha,\beta=1}^{p} \lambda_\alpha \lambda_\beta y_T^{(\alpha)} y_T^{(\beta)} \underbrace{\left(\underline{\phi}_{\underline{\mathbf{x}}^{(\alpha)}}\right)^{\top} \underline{\phi}_{\underline{\mathbf{x}}^{(\beta)}}}_{k(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{x}}^{(\beta)})} + \sum_{\alpha=1}^{p} \lambda_\alpha \stackrel{!}{=} \max_{\{\lambda_\alpha\}} \tag{9.3.39}$$

This suggests the construction of the Gram matrix $\underline{\mathbf{K}}$:

The Gram matrix $\underline{\mathbf{K}}$

$K_{\alpha\beta} = k_{(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{x}}^{(\beta)})}$

	1	2	3	...	j
1	K_{11}	K_{12}	...	...	K_{1j}
2	$\vdots$	$\vdots$	K_{23}	...	K_{2j}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	K_{i1}	K_{i2}	...	...	K_{ij}

Q 9.3.7: *what are the advantages/disadvantages of constructing the matrix* $\underline{\mathbf{K}}$

- Constructing the matrix has the advantage of parallelizing applying the kernel on all the different pairs to speed up computation. However, it requires the storage of $p \times p$ values, which could be costly if the number of training points p is very large.

Q 9.3.8: *What can we say about entries in* $\underline{\mathbf{K}}$ *if we select a radial basis function (RBF)/Gaussian kernel?*

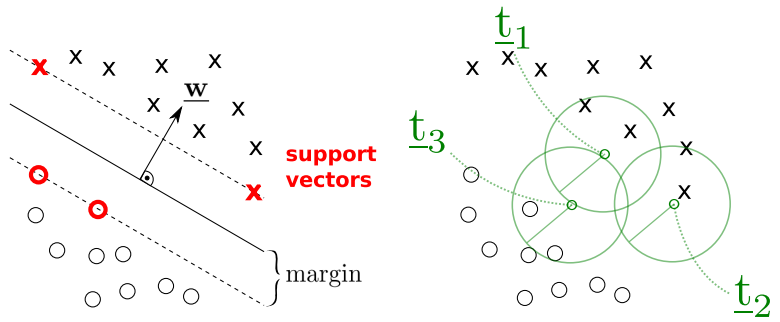
9.3.8.2 SVM-RBF vs. RBF networks

- Recall how an RBF network can be used for classification:

  e.g. 3 centroids , $i = 1, \ldots, 3$

$$y(\underline{\mathbf{x}}) = \text{sign}\left(\sum_{i=1}^{3} w_i \exp\left(-\tfrac{1}{2\sigma_i^2} \|\underline{\mathbf{x}} - \underline{\mathbf{t}}_i\|^2\right)\right)$$

  where $\underline{\mathbf{t}}_i$ denotes the coordinates of the i-th centroid.



(a) SVM for classification for classification

(b) RBF-network for classification

- Comparing this to an SVM with a Gaussian kernel, SVM-RBF:

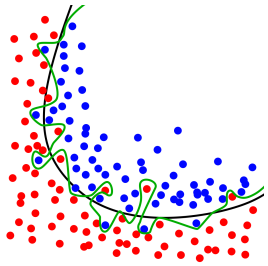
$$y(\underline{\mathbf{x}}) = \text{sign}\Big(\sum_{\alpha\in\text{SV}} \lambda_\alpha y_T^{(\alpha)} \exp\big(-\tfrac{1}{2\sigma^2}\|\underline{\mathbf{x}} - \underline{\mathbf{x}}^{(\alpha)}\|^2\big) + b\Big)$$

Q 9.3.9: *How can we conceptually turn an RBF-network into an SVM-RBF* - A rough outline of doing so would entail:

1. defining as many centroids to match the number of support vectors,
2. placing the centroids on top of the support vectors,
3. find $\underline{\mathbf{w}}$ such that the margin is maximized

9.4 C-SVM

- real-world problems are typically non-separable
- incomplete feature sets & noise
- perfect separation of the training set $\rightsquigarrow$ overfitting



consequences

$$R_{[\underline{\mathbf{w}}]} \quad \leq \quad R^{(p)}_{\mathrm{emp}[\underline{\mathbf{w}}]} \quad + \quad C(p, \mathrm{d}_{\mathrm{VC}})$$

- finite training error $R^{(p)}_{\mathrm{emp}} \neq 0$
- trade-off between the minimization of the training error and the capacity of the model class.

  This is similar to the trade-off when regularizing connectionist neurons with weight decay or LASSO regularization. We no longer assume that minimizing the training error is representative of a good solution, particularly in terms of generalizing to unseen data.

9.4.1 From hard-margin to the soft-margin hyperplane

Hard-margin SVM:

The margin we were constructing for the SVM so far is considered a "hard" in that it prohibits:

1. misclassification error, points falling on the *wrong* side of the decision boundary,

   and

2. points falling *inside* the margin.

This was formulated using the $0-1$ loss as the constraint:

$$y_T^{(\alpha)} \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b \right) \geq 1 \tag{9.4.1}$$

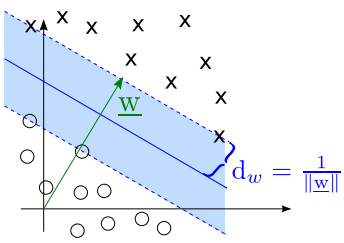


Figure 9.4.1: Hard margin classification.

Soft-margin SVM (C-SVM):

Relaxing the constraint allows for a violating the "rules" of the hard margin classifier to a certain extent, which can be controlled. This is done by introducing a free parameter to the constraint, namely the slack variables $\{\varphi_\alpha\}$. The value of every φ_α is representative of the margin error.

$$y_T^{(\alpha)}\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) \geq 1 - \varphi_\alpha \tag{9.4.2}$$

with $\varphi_\alpha \geq 0$.

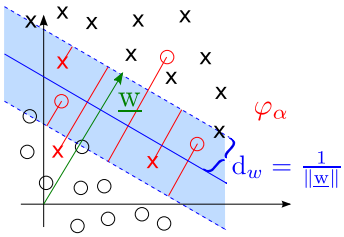


Figure 9.4.2: soft-margin classification.

9.4.1.1 The slack values

Where is the point relative to the margin?	label y_T	prediction $\underline{\mathbf{w}}^\top \underline{\mathbf{x}} + b$	correctly classified	$y_T^{(\alpha)}\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) \geq 1 - \varphi_\alpha$	constraint met for
on	-1	-1	yes	$-1 \cdot (-1) \geq 1 - \varphi_\alpha$	$\varphi_\alpha = 0$
outside	+1	-2	no	$+1 \cdot (-2) \geq 1 - \varphi_\alpha$	$\varphi_\alpha \geq 3$
outside	+1	+20	yes	$+1 \cdot (20) \geq 1 - \varphi_\alpha$	$\varphi_\alpha \geq -19 \Rightarrow$ no slack
on	+1	-1	no	$+1 \cdot (-1) \geq 1 - \varphi_\alpha$	$\varphi_\alpha \geq 2$
inside	+1	-0.1	no	$+1 \cdot (-0.1) \geq 1 - \varphi_\alpha$	$\varphi_\alpha \geq 1.1$
inside	+1	+0.001	yes	$+1 \cdot (0.001) \geq 1 - \varphi_\alpha$	$\varphi_\alpha \geq 0.999$
inside	+1	+0.8	yes	$+1 \cdot (0.8) \geq 1 - \varphi_\alpha$	$\varphi_\alpha \geq 0.2$

Table 2: Example cases for the slack values.

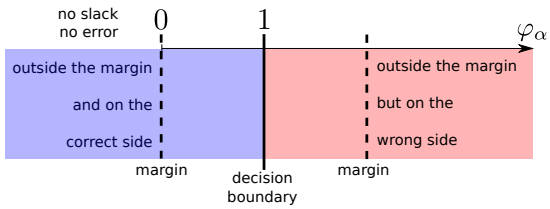


Figure 9.4.3: Overview for interpreting slack values.

- $0 < \varphi < 1$:

  The data point falls *inside* the margin **and** on the *correct* side of the decision boundary

- $\varphi = 1$:

  The data point falls *on* the decision boundary

- $\varphi > 1$:

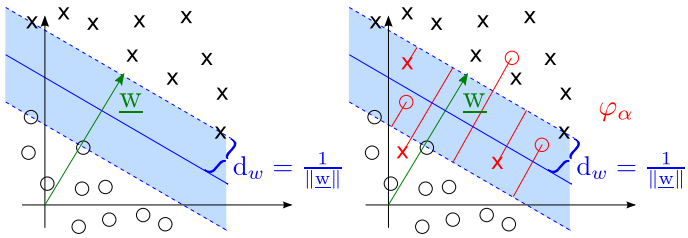
  The data point falls on the *wrong* side of the decision boundary. Whether it's *inside*, *outside* or exactly *on* the margin does not matter.

$$\frac{1}{2}\|\underline{\mathbf{w}}\|^2 + \frac{C}{p}\sum_{\alpha=1}^{p}\varphi_\alpha \stackrel{!}{=} \min \quad \left\{ \begin{array}{l} \text{minimize upper bound on VC dimension} \\ \text{+ minimize (approx.) margin error} \end{array} \right. \tag{9.4.3}$$

constraints ($\forall\alpha$): ($C > 0$: regularization parameter)

$$y_T^{(\alpha)}\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} + b\right) \geq 1 - \varphi_\alpha \qquad \text{normalization \& correct classification of all data points, i.e. "no slack" for } \varphi_\alpha{=}0$$

$$\varphi_\alpha \geq 0 \qquad \text{"margin errors" for } \varphi_\alpha{\neq}0 \text{ "with slack"} \tag{9.4.4}$$



Q 9.4.1: *What do different values for φ_α represent?*

Q 9.4.2: *Why do we have to add the C term?*

Adding The second term to the minimization objective in Eq. (9.4.3) avoids finding the trivial solution where the constrained can be satisfied by making any the slack variable of any point arbitrarily large.

Q 9.4.3: *How does C control complexity?*

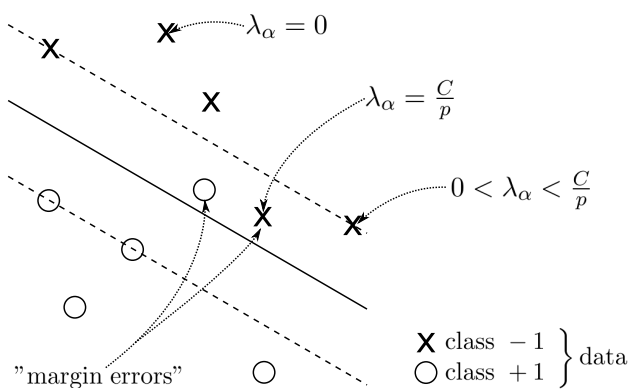
- Objective

$$-\frac{1}{2}\sum_{\alpha,\beta=1}^{p} \lambda_\alpha \lambda_\beta y_T^{(\alpha)} y_T^{(\beta)} \underbrace{\left(\underline{\mathbf{x}}^{(\alpha)}\right)^\top \underline{\mathbf{x}}^{(\beta)}}_{\text{kernel function}} + \sum_{\alpha=1}^{p} \lambda_\alpha \overset{!}{=} \max_{\{\lambda_\alpha\}_{\alpha=1}^{p}} \tag{9.4.5}$$

  See Section 9.3.5 for the derivation. The Lagrangian of the dual problem is the same for SVMs and C-SVMs. However, the difference is in the constraints.

- Constraints:

$$\sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} = 0 \qquad \text{and} \qquad 0 \leq \underbrace{\lambda_\alpha \leq \frac{C}{p}}_{\text{difference to separable case}} \tag{9.4.6}$$



Q 9.4.4: *Where are the support vectors?*

Q 9.4.5: *Does C influence the number of support vectors?*

SVs do not necessarily lie on the margin anymore. The C-SVM has a "soft" margin.

$$\begin{aligned}
\underline{\mathbf{w}} &= \sum_{\alpha=1}^{p} \lambda_\alpha y_T^{(\alpha)} \underline{\mathbf{x}}^{(\alpha)} \qquad \rightsquigarrow \lambda_\alpha \neq 0 \text{ only for support vectors SV} \\
b &= \frac{1}{\#\mathrm{SV}_<} \sum_{\alpha \in \mathrm{SV}_<} \left(y_T^{(\alpha)} - \sum_{\beta \in \mathrm{SV}} \lambda_\beta y_T^{(\beta)} \underbrace{\left(\underline{\mathbf{x}}^{(\beta)}\right)^\top \underline{\mathbf{x}}^{(\alpha)}}_{\text{kernel function}} \right)
\end{aligned}$$

$\mathrm{SV}_<$: SVs with $0 < \lambda_\alpha < \frac{C}{p}$ (SVs on the margin) at which $\varphi_\alpha = 0$ ("no slack").

Further reasoning for why the bias is computed this way is that the bias shifts the hyperplane such that the SVs with zero slack fall on the ± 1 lines parallel to the hyperplane.

Classifier

$$y(\underline{\mathbf{x}}) = \mathrm{sign}\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}} + b\right) = \mathrm{sign}\left(\sum_{\alpha \in \mathrm{SV}} \lambda_\alpha y_T^{(\alpha)} \underbrace{\left(\underline{\mathbf{x}}^{(\alpha)}\right)^\top \underline{\mathbf{x}}}_{\text{kernel function}} + b \right)$$

C-SVM demo: Link to C-SVM applet

9.5 References

References

Christopher J.C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2(2):121–167, Jun 1998. ISSN 1573-756X. doi: 10.1023/A:1009715923555. URL https://doi.org/10.1023/A:1009715923555.

Don Hush and Clint Scovel. On the vc dimension of bounded margin classifiers. *Machine Learning*, 45(1):33–44, 2001. URL https://link.springer.com/content/pdf/10.1023/A:1010971905232.pdf.

Bernhard Scholkopf and Alexander J Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2001.

Vladimir Vapnik. *Statistical learning theory.* Wiley, 1998. ISBN 978-0-471-03003-4.